

Documentação Técnica - Sistema de Emissão de Notas Fiscais

1. Visão Geral

Este documento descreve a solução desenvolvida para o desafio técnico da KORP. O objetivo é apresentar um sistema de emissão de notas fiscais construído utilizando uma arquitetura de microsserviços, focando em simplicidade, código limpo e resiliência (tratamento de falhas).

2. Arquitetura da Solução

O sistema foi dividido em três camadas principais, garantindo o desacoplamento das responsabilidades:

- **Frontend:** Desenvolvido em Angular, responsável pela interface com o usuário.
- **Backend (Microsserviços):** Desenvolvido em Golang. A aplicação foi dividida em dois serviços independentes: Estoque e Faturamento.
- **Banco de Dados:** PostgreSQL, gerenciando os dados de forma relacional.

3. Detalhamento dos Microsserviços

Abaixo estão as responsabilidades de cada serviço construído no backend:

Microsserviço	Porta	Responsabilidade	Endpoints Principais
Serviço de Estoque	8081	Controle de produtos e gerenciamento de saldos.	POST /produtos POST /produtos/{id}/baixar
Serviço de Faturamento	8082	Criação de notas fiscais e rotina de impressão.	POST /notas POST /notas/{id}/imprimir

4. Detalhamento Técnico - Frontend (Angular)

Ciclos de Vida Utilizados

- **ngOnInit:** Utilizado na inicialização dos componentes para requisições de carga de dados iniciais.
- **ngOnDestroy:** Utilizado para realizar o "unsubscribe" de *Observables* abertos e evitar vazamentos de memória (memory leaks).

Uso da Biblioteca RxJS

A biblioteca RxJS foi empregada ativamente no gerenciamento das requisições assíncronas do sistema. O operador **catchError** foi fundamental para capturar falhas oriundas das requisições HTTP e repassar alertas tratados para os componentes de visualização.

Bibliotecas Visuais e Auxiliares

- **Angular Material:** Fornece a base de componentes de UI, como indicadores de carregamento (spinners) para bloquear a tela durante o processamento de impressões.
- **ng-toastr / MatSnackBar:** Empregado para dar feedbacks visuais rápidos (sucesso ou erro) ao usuário.

5. Detalhamento Técnico - Backend (Golang)

Gerenciamento de Dependências

O gerenciamento de dependências da aplicação foi feito nativamente através do **Go Modules**. Cada microsserviço possui seus arquivos go.mod e go.sum independentes.

Frameworks e Bibliotecas

- **Gorilla Mux:** Escolhido pela simplicidade na criação de rotas semânticas e extração de variáveis de URL.
- **database/sql & lib/pq:** Combinação padrão e performática para manipular instruções SQL no banco de dados PostgreSQL.

Nota: Este projeto não utiliza as linguagens C# ou a biblioteca LINQ, sendo 100% desenvolvido em Golang no backend.

6. Resiliência e Tratamento de Falhas

A arquitetura de microsserviços exige cuidado na comunicação entre aplicações. Neste projeto, o requisito obrigatório de falha é tratado da seguinte maneira durante a **Impressão de Notas Fiscais**:

1. O usuário solicita a impressão através do Frontend.
2. O Serviço de Faturamento recebe o pedido, valida se a nota está "Aberta", e faz uma requisição HTTP REST para o Serviço de Estoque solicitando a baixa dos itens.
3. **Cenário de Falha:** Caso o Serviço de Estoque esteja fora do ar, o Serviço de Faturamento captura imediatamente o erro.
4. **Recuperação:** A transação de atualização do status da nota fiscal é cancelada. A nota permanece no status original ("Aberta") garantindo a consistência dos dados.
5. O usuário recebe no frontend uma mensagem clara sobre a falha de comunicação entre os sistemas.